Diabetes Readmission Platform — Project Plan

2026-09-22 · owner @Davide · NOVA IMS EDSB 2026

Executive Summary & Vision

This project builds one platform with two front doors on a shared prediction backend. A **hospital tool** ranks patients at discharge by their risk of inpatient readmission within 30 days, so limited follow-up resources go to those most likely to return. A **patient tool** lets an individual upload their own records, see them in a readable dashboard, and receive plain-language, educational guidance.

The hospital track is the mandatory EDSB 2026 deliverable. The patient track is the innovation layer that lets the work outlive the course as an open-source, deployable product usable by hospitals and, in a limited educational form, by patients.

Both sides are prototypes for education and demonstration. Neither is a certified medical device, and neither replaces a clinician. This framing is deliberate and appears throughout the plan, because a patient-facing health tool carries real safety and legal weight.

Objectives, Scope & Stakeholders

The platform pursues two objectives in priority order: first, deliver the required hospital readmission model; second, wrap it and a patient-data module into a lasting product.

Hospital track (mandatory). At the moment of discharge, estimate each patient's probability of readmission within 30 days, and turn that probability into a ranked, capacity-aware follow-up worklist for the Director of Care Quality and clinical staff.

Patient track (innovation). Let a person upload their own clinical data, visualise it in a dashboard, and read educational, non-diagnostic guidance that contextualises their values against public clinical guidelines.

Stakeholder	What they need from the platform
Director of Care Quality	A defensible way to prioritise scarce follow-up capacity
Clinicians / discharge staff	Per-patient risk with reasons they can trust and act on

Stakeholder	What they need from the platform
Patients	Their own data made understandable, plus safe next-step suggestions
Course evaluators	A sound, reproducible ML pipeline and clear reasoning

In scope: readmission risk model; risk-tier logic tied to follow-up capacity; hospital worklist UI; patient upload, dashboard and guardrailed guidance; deployment, documentation and licensing.

Out of scope: any diagnostic or treatment decision; real patient data during the course; integration with real hospital systems (EHR/HL7); regulatory certification. These are named as future work, not deliverables.

Data Foundation & Core ML Task

The task is binary classification of a rare event: `readmitted = <30` is the positive class (~11% of encounters); `>30` and `NO` are negative. The data is the UCI Diabetes 130-US Hospitals set: 101,766 encounters, 71,518 patients, 1999-2008.

Data-prep decisions that make or break this dataset:

1. **Split by patient, not by encounter.** `patient_nbr` repeats, so a random encounter split leaks the same patient into train and test. Either keep only the first encounter per patient (Strack et al.) or use a grouped split on `patient_nbr`.
2. **Remove death and hospice discharges.** `discharge_disposition_id` in {11, 13, 14, 19, 20, 21} means the patient died or entered hospice and cannot be readmitted. Leaving them in poisons the label.
3. **Use only features known at discharge.** Every field is from the completed encounter, which is valid, but the report must state this explicitly.
4. **Decode and group.** Decode the three ID lookups from `IDS_mapping.csv`; group `diag_1/2/3` into the Strack ICD-9 categories; treat `?` as missing per field.
5. **Handle missingness and constants.** Drop `weight` (96.86% missing) and constant meds (`examide`, `citoglipton`); keep `payer_code` and `medical_specialty` as categories with an explicit "Unknown" level.

Modelling. Baseline logistic regression (interpretable), then gradient-boosted trees (XGBoost / LightGBM). Address the ~11% imbalance with class weights rather than naive oversampling.

Evaluation that fits the goal. Because the output ranks patients for scarce follow-up, report PR-AUC and ROC-AUC, **recall and precision at top-K** (K = follow-up capacity), a **calibration plot** with post-hoc calibration, and a **decision curve**. Convert calibrated probabilities into High / Medium / Low tiers whose cutoffs are set by capacity, not by a default 0.5 threshold.

System Architecture

One backend serves both audiences, which keeps the model in a single place and avoids two divergent codebases. A FastAPI service loads the serialized scikit-learn pipeline (preprocessing + model + calibration) and exposes scoring; two separate UIs call it with role-appropriate screens.

```
flowchart TD
  subgraph Core[Shared backend]
    API[FastAPI prediction service]
    PIPE[pipeline.joblib<br/>prep + model + calibration]
    GUIDE[(Clinical guidelines<br/>knowledge base)]
    API --> PIPE
  end
  subgraph Hospital[Hospital track]
    HW[Discharge worklist<br/>+ capacity slider]
    HR[Per-patient risk<br/>+ SHAP reasons]
  end
  subgraph Patient[Patient track]
    PU[Upload own data]
    PD[Personal dashboard]
    PA[Educational guidance<br/>LLM + guardrails]
  end
  HW --> API
  HR --> API
  PU --> PD
  PD --> PA
  PA --> GUIDE
  PA -. optional context .-> API
```

The hospital UI is a ranked worklist plus a per-patient view with SHAP explanations. The patient UI ingests an upload, renders a dashboard, and passes structured values to the guidance layer, which is grounded in a curated guidelines knowledge base rather than free generation. The readmission model is only an optional context signal on the patient side, never the source of personal advice (see next section).

Patient Module & the LLM Decision

Do you need an LLM? Yes, but only as a grounded explanation layer, not as the brain of the feature. The valuable, safe job for an LLM here is to read structured values a patient enters, explain what they mean in plain language, and surface guidance retrieved from trusted sources. It should not diagnose, and it should not invent thresholds.

Why the readmission model cannot generate patient advice. That model answers one narrow question for hospitals: probability of readmission within 30 days for inpatients discharged from a specific US network in 1999-2008. It is the wrong population and era for a present-day individual, it is correlational (the brief states it does not establish causation), so "lower your risk" cannot be read off its features. The patient track therefore rests on published clinical guidelines, with the risk score at most an optional talking point.

Recommended guardrailed pipeline for the patient side:

1. **Structured intake.** Parse the upload into typed fields (e.g. HbA1c, glucose, blood pressure). Prefer explicit inputs or parsed lab values over free text; if you accept PDFs, extract to structured fields first.
2. **Deterministic dashboard.** Compute ranges, trends and flags in code, not in the LLM, so the numbers are always correct and reproducible.
3. **Retrieval over a curated knowledge base (RAG).** Index vetted guideline summaries (e.g. WHO, ADA, NICE). The LLM answers only from retrieved passages and cites them.
4. **Constrained generation.** System prompt fixes scope, tone and refusals; the model explains and suggests general, guideline-based next steps, never a diagnosis or medication change.
5. **Hard guardrails.** Detect red-flag values and route to "seek professional care now"; block out-of-scope questions; log every answer with its sources.
6. **Persistent disclaimer.** Every screen states this is educational, not medical advice, and does not replace a clinician.

A smaller, cheaper model with good retrieval usually beats a large model left to free-associate. The safety of this feature comes from retrieval and guardrails, not from model size.

Responsible AI, Privacy & Regulatory Considerations

A patient-facing health tool raises the bar well above a normal course project, so the plan treats safety and privacy as first-class deliverables, not afterthoughts.

Fairness. Report model performance separately across `race`, `gender` and `age` bands, and flag any group where the model under-performs. Note the three `Unknown/Invalid` gender rows are too few for reliable group metrics.

Honesty about the model. State plainly that the score is correlational and observational, that follow-up is prioritised by risk but not proven to reduce readmission, and that probabilities are calibrated so a "20% risk" means roughly what it says.

Privacy (GDPR). Health data is special-category data under GDPR Article 9, requiring explicit consent and data minimisation. For the course, use only the public UCI dataset for modelling and, on the patient demo, process uploads in-session with no server-side storage by default; never use real patient data.

Regulatory framing. Software that supports clinical or health decisions can qualify as a medical device under EU MDR 2017/745; a patient-advice tool could fall in scope. The prototype avoids this by being explicitly educational and non-diagnostic, with disclaimers on every screen. Real deployment to hospitals or patients would require clinical validation, regulatory review and a data-protection assessment.

What real-world use would need (future work): prospective clinical validation, EHR integration, a named clinical owner, ongoing monitoring, and formal MDR/GDPR compliance. Listing these honestly is a strength of the report, not a weakness.

Technical Stack & Deployment

Everything is open-source with a free or low-cost hosting path, so the platform can stay online after the course with no budget.

Layer	Choice
Data & modelling	Python, pandas, scikit-learn, XGBoost / LightGBM
Explainability	SHAP
Experiment tracking	MLflow (optional)
Backend	FastAPI serving `pipeline.joblib`
UIs	Streamlit (fastest) or a light React front-end
Patient guidance	Small LLM + retrieval (RAG) over a curated guideline index
Packaging	Docker
Hosting	Hugging Face Spaces / Streamlit Community Cloud / Render

The key packaging rule: serialize the **entire pipeline** (preprocessing + model + calibration) into one `pipeline.joblib`, so training and inference can never drift apart and the API loads a single object.

```
diabetes-readmission/
  data/            raw / interim / processed  (raw gitignored)
  notebooks/       01_eda  02_prep  03_model  04_shap
  src/             data_prep.py  features.py  train.py  predict.py
  models/          pipeline.joblib
  api/             FastAPI app  (POST /predict)
  app_hospital/    Streamlit worklist + risk view
  app_patient/     Streamlit dashboard + guidance
  guidelines/      curated KB for retrieval
  requirements.txt  Dockerfile  LICENSE  README.md
```

Roadmap, Phases & Milestones

The sequence is deliberate: finish and lock the mandatory hospital deliverable before investing in the patient innovation layer, so the graded requirement is never at risk.

Phase	Focus	Milestone / definition of done
1	Data prep & EDA	Clean, patient-split dataset; decoded IDs; documented decisions

Phase	Focus	Milestone / definition of done
2	Hospital model	Calibrated model; PR-AUC, recall@top-K, SHAP; risk tiers
3	Hospital MVP	FastAPI + Streamlit worklist with capacity slider, deployed
4	Patient module	Upload, deterministic dashboard, guardrailed RAG guidance
5	Responsible AI	Fairness audit, disclaimers, privacy notes written up
6	Hardening & docs	Docker, README, license, demo script, final report & slides

Phases 1-3 are the graded core and should be treated as a shippable product on their own. Phase 4 onward can grow after the course. Freeze the modelling dataset early so results stay reproducible across the team.

Team Roles & Work Split

Work splits into five streams that run partly in parallel once the shared dataset is frozen after Phase 1. With a team of four, one person can hold two lighter streams, and everyone contributes to the report.

Stream	Owns	Runs in parallel after
Data & ML	Prep, features, model, calibration, evaluation	Phase 1
Backend / API	FastAPI service, pipeline packaging, Docker	Phase 2
Hospital UI	Worklist, capacity slider, SHAP views	Phase 2
Patient UI & LLM	Upload, dashboard, RAG guidance, guardrails	Phase 3
Responsible AI & report	Fairness, privacy, disclaimers, slides, write-up	Phase 2

Shared checkpoints keep the streams aligned: freeze the dataset and metric definitions after Phase 1, agree the API contract before UIs start, and hold a demo dry-run before the final submission. Data & ML and Responsible AI make a natural pairing for one person,

since both feed the report.

Sustainability Beyond the Course & Risks

For the platform to outlive EDSB 2026, treat it as a product from day one: a public, well-documented, containerized repository that a stranger can clone and run. Concretely, that means a clear README with a one-command Docker start, a permissive license, a versioned model in a registry, modular hospital and patient services that can evolve independently, and a short recorded demo. This is what turns a course submission into something a hospital team or an open-source contributor could actually pick up.

Risk	Mitigation
Scope creep from the patient track	Ship hospital Phases 1-3 first; gate Phase 4 behind that
Regulatory / ethical exposure of patient advice	Educational framing, RAG grounding, guardrails, disclaimers
Over-reliance on a 1999-2008 dataset	State era/population limits; never present it as current guidance
Data leakage inflating metrics	Patient-level split; drop death/hospice; discharge-only features
Privacy of any real upload	No real patient data; in-session processing; no default storage
Team drift across streams	Freeze dataset and API contract early; shared checkpoints

The single most valuable sustainability move is honest documentation: a README that states what the tool is, what it is not, and what real-world use would require. That is also what a reviewer and a future maintainer will judge first.